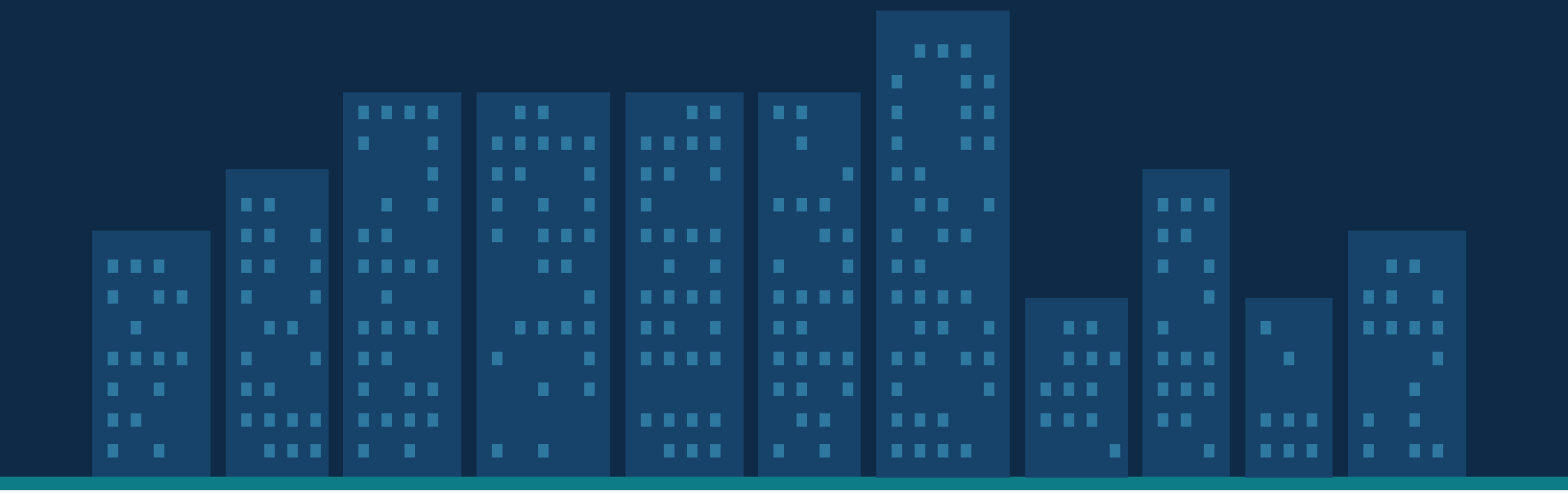
Conjunto Inteligente

Arquitectura de referencia, auditoría técnica
y plan de ingeniería para un gemelo digital
de conjunto residencial multi-edificio



DOCUMENTO TÉCNICO · REVISIÓN DE LOS MÓDULOS A–I

Versión 1.0 · 27 de octubre de 2026

Contenido: estado verificado, requisitos de escala, arquitectura objetivo, modelo de datos, ingesta federada, visor 3D, seguridad, operación, auditoría de código y hoja de ruta.

Revisión por lectura del código propuesto; los fragmentos de referencia están marcados según su estado de ejecución.

Contenido

1. Resumen ejecutivo . . . 4

1.1 Qué cambia cuando son muchos edificios . . . 4

1.2 Cinco decisiones que lo desbloquean todo . . . 4

1.3 Lectura de la auditoría . . . 4

2. Qué está verificado y qué no . . . 5

3. De demo a conjunto: requisitos de escala . . . 6

3.1 Tamaño del árbol por escenario . . . 6

3.2 Principio de diseño: nodo solo si se opera o se navega . . . 6

3.3 Presupuesto de telemetría . . . 6

4. Arquitectura objetivo . . . 7

4.1 Principios . . . 7

4.2 Responsabilidades por módulo . . . 7

4.3 Jerarquía y taxonomía de nodos . . . 8

5. Modelo de datos para el conjunto . . . 9

5.1 Cómo consultar el subárbol . . . 9

5.2 Esquema propuesto (PostgreSQL, no ejecutado) . . . 9

5.3 Convenciones de coordenadas . . . 10

6. Ingesta federada de modelos IFC . . . 11

6.1 Decisiones clave . . . 11

6.2 Casos límite que el pipeline debe manejar . . . 11

7. Visor 3D a escala . . . 13

7.1 Niveles de detalle y carga bajo demanda . . . 13

7.2 Técnicas imprescindibles . . . 13

7.3 Presupuesto de rendimiento . . . 13

8. Seguridad multi-tenant y RBAC . . . 14

8.1 Modelo de tenancy y alcance . . . 14

8.2 Matriz de permisos propuesta . . . 14

8.3 Reglas de oro . . . 14

8.4 Política RLS (referencia, no ejecutada) . . . 14

9. Telemetría, reglas y alertas a escala . . . 15

9.1 Flujo recomendado . . . 15

9.2 Almacenamiento y retención (punto de partida) . . . 15

10. Operación y producción 16

10.1 Objetivos de servicio propuestos 16

10.2 Copias de seguridad (3-2-1) 16

10.3 Despliegue y CI/CD 16

11. Auditoría del código propuesto (módulos A–I) 17

A · Ingesta IFC 17

B · Visor 3D 18

E · Catálogo de activos 18

G · Auth multi-tenant y RBAC 19

H · Reportes 19

I · Observabilidad y hardening 20

12. Correcciones de referencia 21

12.1 Identidad y tenant (cierra G-01, G-02, G-03, G-08, G-11) 21

12.2 CSV seguro (cierra H-01, H-04) · probado 21

12.3 Reportes programados que respetan el cron (cierra H-02) 21

12.4 Ingesta por clave de S3 y engine apto para workers (cierra A-01, A-10) 22

12.5 Hook de hijos y selectores por nodo (cierra B-01, B-02) 22

12.6 Métricas (cierra I-01, I-05) 22

12.7 Prueba automática de fuga entre tenants 22

13. Hoja de ruta por fases 23

14. Riesgos y decisiones abiertas 24

14.1 Registro de riesgos 24

14.2 Decisiones por tomar 24

Anexo A. Lista de salida a producción 25

Anexo B. Glosario 26

1. Resumen ejecutivo

Tu observación es correcta: lo construido hasta ahora es una base sólida para **un edificio de demostración**, no para un **conjunto de edificios**. Un conjunto no es “el mismo edificio, pero más grande”: cambia la arquitectura de datos, la forma de cargar lo 3D, la ingesta de modelos y el aislamiento entre clientes. Este documento define esa arquitectura, audita el código propuesto en los módulos A–I y fija un plan por fases con criterios de salida medibles.

52	2	12	63.025
hallazgos en el código propuesto	críticos (autenticación)	altos	nodos en el conjunto grande de referencia

1.1 Qué cambia cuando son muchos edificios

- **Escala del árbol.** Lo probado fue un árbol de 5.481 nodos. Un conjunto mediano ronda 13.361 y uno grande 63.025, todavía sin contar áreas comunes, parqueaderos ni sistemas.
- **Fuentes múltiples.** Cada torre llega en varios archivos IFC (arquitectura, estructura, MEP) con sus propias coordenadas: hay que federarlos, no importarlos uno a uno.
- **Navegación por niveles.** El usuario abre el conjunto y baja a torre, piso y activo. Las consultas deben agregar (rollups) en lugar de recorrer miles de nodos en cada vista.
- **Operación transversal.** Alertas y reportes por torre, por zona y para todo el conjunto; sistemas que cruzan torres (red hidráulica, planta eléctrica, ascensores).
- **Seguridad con alcance.** Más roles (administración, vigilancia, residente) y permisos por subárbol, no solo por tenant.

1.2 Cinco decisiones que lo desbloquean todo

1. **El gemelo es un árbol curado, no un volcado del IFC.** Miles de nodos (espacios y activos operables); los cientos de miles de elementos geométricos viven en tiles, sin fila en la base de datos.
2. **Ingesta federada e idempotente.** Un *model_source* por archivo y disciplina, clave natural (tenant, modelo, GlobalId) y reingestas seguras.
3. **Visor por niveles de detalle con streaming.** Conjunto → torre → piso → activo, con instancing y picking por identificador.
4. **Aislamiento verificable.** Tenant solo desde el token firmado, membresía comprobada, RLS en PostgreSQL y pruebas automáticas de fuga entre tenants.
5. **Verificar contra el stack real antes de sumar módulos.** PostgreSQL/TimescaleDB, al menos tres torres en IFC reales, backend levantado y frontends compilados.

1.3 Lectura de la auditoría

Se revisaron 52 puntos en seis módulos: 2 críticos, 12 altos, 30 medios y 8 bajos. La mayoría son correcciones puntuales; dos áreas requieren **rediseño**: la geometría (A-06, B-05) y el modelo de datos de la jerarquía. Los dos críticos están en autenticación y tenancy y deben cerrarse primero (capítulo 11).

> **Alcance y método**
> Revisión **por lectura** de los módulos A, B, E, G, H, I (C y D no estaban en el material). Solo se reprodujeron en ejecución el error del CSV y el conteo de nodos. Donde hay duda se indica “verificar”.

2. Qué está verificado y qué no

Para decidir con honestidad cuánto falta, se usa una escala de madurez. Las marcas ✓ de resúmenes anteriores significan **“módulo redactado”** (L0), no “probado”.

Nivel	Significado	Evidencia mínima
L0	Redactado	Código propuesto; nadie lo ha ejecutado.
L1	Ejecuta aislado	Se importa y corre una vez sin errores.
L2	Pruebas automáticas	Suite que pasa, con dobles o base sustituta (SQLite, servidor simulado).
L3	Integrado en el stack real	Pasa contra PostgreSQL/TimescaleDB, Redis, MinIO y backend levantado.
L4	Probado con datos reales y carga	IFC reales, volumen del conjunto objetivo, métricas medidas.

Componente	Evidencia disponible	Nivel	Para subir de nivel
Backend ERP y núcleo del gemelo	200 pruebas con runner offline; 313 claves foráneas con 0 hallazgos.	L2	Migraciones y pruebas contra PostgreSQL real.
Jerarquía SQL (rollup, paginación, ciclos, búsqueda con ruta, vecinos)	7 pruebas con el SQL real en SQLite sobre 5.481 nodos, comparadas con un oráculo de fuerza bruta (0,00 a 0,19 s por prueba).	L2	Ejecutar las mismas consultas en PostgreSQL y medir con EXPLAIN a 63 mil nodos.
bim-command-center.html (43 KB)	12 pruebas E2E en Chromium con servidor simulado; CSP con hashes verificado.	L2	Probar contra backend levantado.
Otros frontends (carcasa, maquetas, React, ERP)	Solo sintaxis revisada; React sin node_modules; ERP sin llamadas de red.	L0–L1	Instalar, compilar y conectar.
Visor 3D (three + web-ifc)	No ejecutado: faltan librerías y un IFC real.	L0	Vendorizar dependencias y probar con IFC reales.
A · Ingesta IFC	Código propuesto.	L0	Rediseño de geometría y jobs (cap. 6).
B · Visor R3F	Código propuesto.	L0	LOD, instancing y selectores (cap. 7).
E · Catálogo de activos	Código propuesto.	L0	Aislamiento por tenant y RBAC.
G · Auth multi-tenant y RBAC	Código propuesto; 2 defectos críticos.	L0	Cap. 8 y 12.1.
H · Reportes	Código propuesto; 1 error reproducido.	L0	Cap. 12.2 y 12.3.
I · Observabilidad y hardening	Código propuesto.	L0	Cap. 10.
C · Reglas y D · Editor de muros	Marcados ✓ en el resumen; el código no estaba en el material revisado.	N/D	Revisión pendiente.

Conclusión
Hoy ningún componente supera L2 y nada ha corrido contra el stack completo. Eso no invalida el trabajo: significa que el siguiente paso de mayor valor no es otro módulo, sino **subir lo existente a L3** (fase F0 del plan).

3. De demo a conjunto: requisitos de escala

3.1 Tamaño del árbol por escenario

La cifra de nodos se calcula como 1 + torres + pisos + unidades + activos. Los escenarios mediano y grande son hipótesis de trabajo para dimensionar; deben reemplazarse por los conteos reales del conjunto.



Nodos del gemelo = 1 + torres + pisos + unidades + activos (sin áreas comunes ni sistemas).

Escenario	Torres	Pisos por torre	Unidades por piso	Activos por unidad	Nodos
Piloto (probado)	8	12	8	6	5.481
Conjunto mediano	10	15	8	10	13.361
Conjunto grande	24	25	8	12	63.025

Áreas comunes, parqueaderos, cuartos técnicos y sistemas se suman aparte. Los nodos no son los elementos IFC: un modelo completo de una torre contiene muchos más elementos geométricos que activos operables (confirmar el orden de magnitud con los IFC reales).

3.2 Principio de diseño: nodo solo si se opera o se navega

Se convierte en nodo	Se queda como geometría (tile)
Conjunto, zona, torre, piso, unidad, espacios relevantes.	Muros, losas, columnas, cielorrasos, acabados.
Equipos operables: bombas, ascensores, tableros, válvulas principales, ventiladores.	Tubería y ductos como volumen (segmentos sin estado propio).
Sensores y actuadores con telemetría.	Mobiliario fijo y elementos decorativos.

Así el árbol sigue siendo del orden de decenas de miles y la base de datos se mantiene ágil.

3.3 Presupuesto de telemetría

Hipótesis: un sensor por cada cuatro activos. La tasa depende del intervalo de muestreo; la tabla muestra dos casos.

Escenario	Activos	Sensores	Mensajes/s a 30 s	Mensajes/s a 5 s	Filas por día a 30 s
Piloto	4.608	1.152	38,4	230,4	3.317.760
Mediano	12.000	3.000	100	600	8.640.000
Grande	57.600	14.400	480	2.880	41.472.000

Con decenas de millones de filas por día se necesitan hypertables, compresión, retención por capas y agregados continuos; y las reglas deben evaluarse con estado en memoria, no consultando la base por mensaje (cap. 9).

4. Arquitectura objetivo

4.1 Principios

- **Monolito modular + workers.** Un equipo pequeño gana más con transacciones simples y un despliegue único que con microservicios; la telemetría ya se separa como servicio propio.
- **Geometría fuera de la base de datos.** PostgreSQL guarda el árbol y las relaciones; S3/MinIO guarda tiles y archivos; el manifiesto los vincula.
- **Defensa en profundidad para tenants.** Token firmado, membresía por petición y RLS en la base de datos.
- **Pipelines idempotentes.** Cualquier ingesta, reporte o cálculo puede repetirse sin duplicar ni corromper.
- **Contrato primero.** OpenAPI como fuente de verdad; pruebas de contrato generadas desde él.
- **Observabilidad desde el día cero.** Métricas, trazas y logs con *tenant_id* en cada evento.

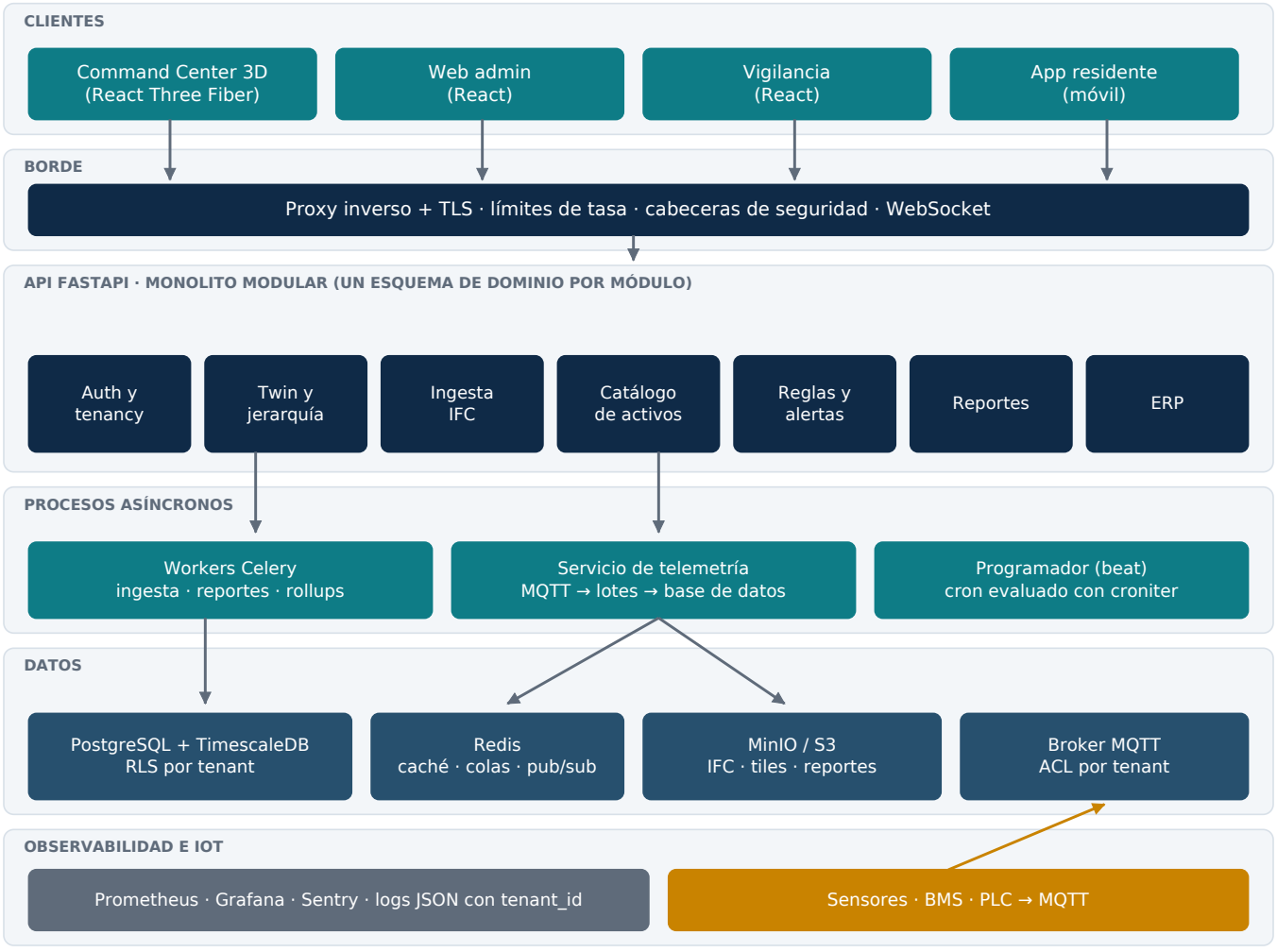


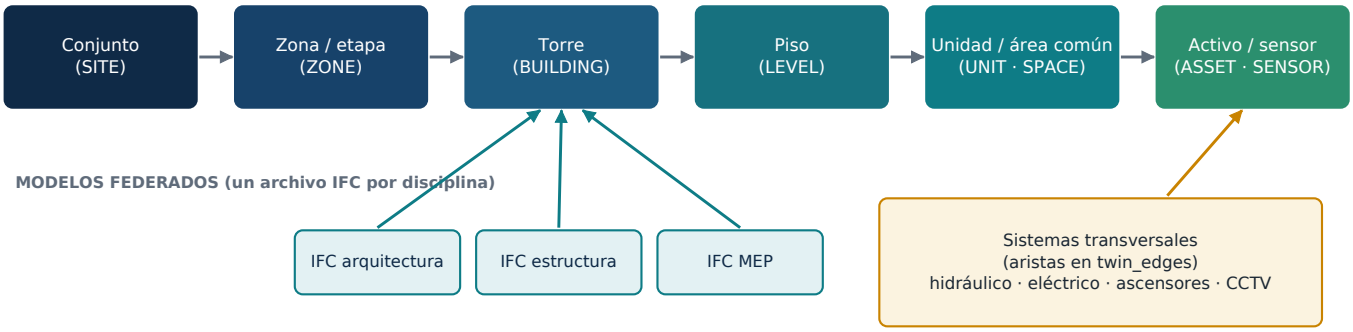
Figura 1. Arquitectura objetivo por capas. Las flechas indican el sentido dominante de la dependencia.

4.2 Responsabilidades por módulo

Módulo	Responsabilidad	Datos que posee	No debe hacer
Auth y tenancy	Identidad, membresías, roles, tokens, auditoría.	users, tenants, user_tenant, audit_log	Decidir permisos de negocio de otros módulos.

Módulo	Responsabilidad	Datos que posee	No debe hacer
Twin y jerarquía	Árbol, aristas, rollups, búsqueda con ruta.	twin_nodes, twin_edges, twin_node_rollup	Servir geometría pesada.
Ingesta IFC	Jobs, validación, extracción, geometría, upsert.	model_sources, ingest_jobs, geometry_tiles	Bloquear peticiones HTTP.
Catálogo	Plantillas e instanciación de activos.	asset_templates	Escribir fuera del tenant.
Reglas y alertas	Evaluar telemetría, ciclo de vida de alertas.	rules, alerts	Consultar la base por mensaje.
Reportes	Generación sincrónica pequeña, asíncrona grande, programada.	scheduled_reports, report_runs	Recorrer el árbol sin rollups ni ltree.
Telemetría	Consumir MQTT, validar, insertar por lotes.	sensor_telemetry (hypertable)	Aplicar lógica de negocio.

4.3 Jerarquía y taxonomía de nodos



El árbol (parent_id) responde “dónde está”; el grafo (twin_edges) responde “de qué depende”.

Figura 2. Árbol espacial, modelos federados por torre y sistemas transversales como grafo.

node_type	Equivale a	Origen típico	Notas
SITE	Conjunto residencial	IfcSite o creación manual	Raíz de la jerarquía; un tenant puede tener varios.
ZONE	Etapa, manzana o supermanzana	Manual	Opcional; agrupa torres para reportes y permisos.
BUILDING	Torre o bloque	IfcBuilding	Ancla de un model_source por disciplina.
LEVEL	Piso, sótano, cubierta	IfcBuildingStorey	Unidad de carga de geometría (tile).
UNIT	Apartamento o local	IfcSpace con clasificación	Alcance natural de un residente.
SPACE	Zona común, cuarto técnico, parqueadero	IfcSpace	Puede no tener geometría propia.
ASSET / SENSOR	Equipo operable o sensor	Lista blanca de clases IFC o catálogo	Recibe estado y telemetría.

5. Modelo de datos para el conjunto

5.1 Cómo consultar el subárbol

Hoy la jerarquía se apoya en *parent_id* y CTE recursivas. Esa base es correcta (las 7 pruebas SQL lo respaldan) pero cada rollup recorre el subárbol completo. A 63 mil nodos y con reportes, filtros y alertas por torre, conviene añadir una ruta materializada y agregados precalculados.

Opción	Lectura de subárbol	Escritura	Integridad	Cuándo usarla
parent_id + CTE recursiva (actual)	Correcta; el costo crece con el subárbol.	Trivial.	Alta: una sola fuente.	Navegación por niveles.
parent_id + path (ltree)	Rápida con índice GiST: `path <@ 'a.b'`.	Mover un subárbol reescribe rutas.	Media: la ruta debe mantenerse.	Reportes y filtros por torre o zona.
Tabla de cierre	Muy rápida.	N filas por inserción.	Media.	Si domina la lectura y hay poca mutación.
Rollups materializados	O(1) para estados agregados.	Actualizar ancestros al cambiar el estado.	Eventual.	Vista de conjunto y de torre.

Recomendación
Mantener *parent_id* como fuente de verdad, añadir *path* (ltree) y *depth*, y una tabla *twin_node_rollup* actualizada por workers. Validar la elección con **EXPLAIN ANALYZE en PostgreSQL con 63 mil nodos**: SQLite confirma la lógica, no el plan de ejecución.

5.2 Esquema propuesto (PostgreSQL, no ejecutado)

```
CREATE EXTENSION IF NOT EXISTS ltree;

CREATE TABLE model_sources (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id       uuid NOT NULL,
  building_node_id uuid REFERENCES twin_nodes(id),   -- torre a la que se acopla
  discipline      text NOT NULL,                     -- arq | est | mep | urb
  ifc_schema      text,                              -- IFC2X3 | IFC4 | IFC4X3
  file_key        text NOT NULL,                     -- clave en S3
  sha256          text NOT NULL,
  origin_m        double precision[3],               -- desplazamiento al origen del sitio
  crs             text,                              -- sistema de coordenadas, si existe
  revision        int  NOT NULL DEFAULT 1,
  created_at      timestamptz NOT NULL DEFAULT now(),
  UNIQUE (tenant_id, sha256)
);

ALTER TABLE twin_nodes
  ADD COLUMN path            ltree,
  ADD COLUMN depth           smallint,
  ADD COLUMN global_id       text,
  ADD COLUMN model_source_id uuid REFERENCES model_sources(id),
  ADD COLUMN retired_at      timestamptz;

CREATE UNIQUE INDEX uq_node_gid ON twin_nodes (tenant_id, model_source_id, global_id)
  WHERE global_id IS NOT NULL;
CREATE INDEX ix_node_parent ON twin_nodes (tenant_id, parent_id);
CREATE INDEX ix_node_type   ON twin_nodes (tenant_id, node_type);
CREATE INDEX ix_node_path   ON twin_nodes USING gist (path);

CREATE TABLE twin_node_rollup (
  node_id          uuid PRIMARY KEY REFERENCES twin_nodes(id) ON DELETE CASCADE,
  tenant_id        uuid NOT NULL,
  descendant_count int  NOT NULL DEFAULT 0,
  ok int NOT NULL DEFAULT 0, warning int NOT NULL DEFAULT 0,
  critical int NOT NULL DEFAULT 0, offline int NOT NULL DEFAULT 0,
  updated_at       timestamptz NOT NULL DEFAULT now()
```

```
);

CREATE TABLE geometry_tiles (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(), tenant_id uuid NOT NULL,
  model_source_id uuid REFERENCES model_sources(id), node_id uuid REFERENCES twin_nodes(id),
  lod smallint NOT NULL, file_key text NOT NULL, bytes bigint, sha256 text NOT NULL,
  bbox double precision[6]                          -- minx,miny,minz,maxx,maxy,maxz
);
```

5.3 Convenciones de coordenadas

- Metros, eje Y hacia arriba, transformaciones **locales al padre** (un único lugar donde se aplica cada transformación).
- Cada torre tiene origen propio; el desplazamiento al origen del sitio y el CRS se guardan en *model_sources*, no en los vértices.
- Las coordenadas proyectadas grandes producen temblor por precisión de coma flotante: se restan antes de exportar la geometría.
- El visor aplica la transformación una sola vez; la ingesta no “hornea” posición global en la malla (ver A-04).

6. Ingesta federada de modelos IFC

El pipeline sustituye el flujo “subir bytes → tarea Celery → un GLB por elemento” por un proceso por etapas, observable y repetible.



Figura 3. Pipeline de ingesta por etapas.

6.1 Decisiones clave

- **Subida directa a S3.** La API entrega una URL prefirmada y registra la clave. El archivo nunca pasa por la API ni por el broker.
- **Un job por archivo.** Estados queued, running, failed, completed; progreso y errores por elemento; reintentos seguros.
- **Transacción por modelo.** Si un archivo falla, no se toca el estado anterior del gemelo.
- **Lista blanca de clases operables.** Configurable por tenant: qué clases IFC se vuelven ASSET o SENSOR.
- **Geometría fusionada.** Mallas por piso y material, instancias para familias repetidas y un manifiesto GlobalId → tile + rango o instancia para el picking.
- **Federación.** Varios *model_sources* se acoplan a la misma torre; los conflictos se resuelven por prioridad de disciplina y se reportan.

6.2 Casos límite que el pipeline debe manejar

Caso	Efecto si no se maneja	Respuesta
IFC sin IfcBuildingStorey	Pisos ausentes; activos huérfanos.	Crear un nivel “Sin nivel” y advertir.
GlobalId duplicado dentro del mismo archivo	Colisión en el upsert.	Rechazar el elemento y registrarlo en ingest_job_items.
Unidades en milímetros o pies	Modelo con escala errónea.	Leer IfcUnitAssignment y normalizar a metros.
Coordenadas proyectadas muy grandes	Temblor visual por precisión.	Origen local por torre; guardar el desplazamiento aparte.
Elementos sin geometría	Nodos sin forma en el visor.	geometry_state = none; usar el volumen del espacio si hay perfil.
Archivo corrupto o esquema no soportado	Job colgado o datos a medias.	Estado failed con mensaje; sin cambios en el gemelo.

Caso	Efecto si no se maneja	Respuesta
Reexportación con cambios	Nodos que desaparecen y pierden historial.	retired_at y conservar telemetría y alertas.
Modelos muy grandes	Memoria del worker agotada.	Procesar por contenedor espacial; límites de memoria y concurrencia.

Prueba de aceptación de la fase
Ingestar tres torres reales (arquitectura + MEP) dos veces seguidas y obtener **cero nodos duplicados**, con el tiempo de ingesta medido por torre. Ese número fija el objetivo de rendimiento; hoy no existe ninguna medición.

7. Visor 3D a escala

7.1 Niveles de detalle y carga bajo demanda

Nivel	Qué se muestra	Fuente de datos	Objetivo (a validar)
L0 Conjunto	Implantación, torres como masas simplificadas, zonas comunes, estado agregado por torre.	Un tile ligero por torre (pocos miles de triángulos) + rollups.	Primera imagen en ≤ 3 s
L1 Torre	Fachada, losas por piso, estado agregado por piso.	Tile de fachada + manifiesto de pisos.	Entrar en ≤ 2 s
L2 Piso	Muros, espacios, unidades; activos como instancias.	Un tile fusionado por piso.	Entrar en ≤ 1,5 s
L3 Activo	Geometría detallada, telemetría e historial.	GLB individual bajo demanda.	Abrir en ≤ 0,5 s

Política de carga: solo se mantiene en memoria el subárbol seleccionado y sus vecinos inmediatos; al salir de un nivel se liberan geometrías, materiales y texturas (*dispose*). Los archivos de tiles llevan hash en el nombre para cachearse como inmutables.

7.2 Técnicas imprescindibles

- **Fusión estática por piso y material** para reducir draw calls; **InstancedMesh** para luminarias, sensores y equipos repetidos.
- **Picking por identificador** (mapa instancia → nodo) y raycast con BVH, en lugar de un manejador de eventos por malla.
- **Resaltado y estado por atributo** o shader, sin clonar materiales.
- **Selectores de Zustand por nodo** y telemetría agrupada a 4–10 Hz; nunca suscribir un componente al estado completo.
- **Compresión:** Draco o meshopt para geometría y KTX2 para texturas; decodificación en Web Workers.
- **Cámara:** transiciones puntuales que se cancelan cuando el usuario interactúa (no un lerp permanente).
- **Alternativas a evaluar:** formatos orientados a fragmentos de IFC o 3D Tiles si los modelos superan lo manejable con glTF por tiles.

7.3 Presupuesto de rendimiento

Indicador (hipótesis a calibrar)	Escritorio	Móvil
Draw calls por fotograma	≤ 1.000	≤ 300
Triángulos visibles	≤ 3 millones	≤ 500 mil
Memoria de geometría y texturas	≤ 800 MB	≤ 250 MB
Fotogramas por segundo	≥ 45	≥ 30
Tiempo a primera imagen del conjunto	≤ 3 s (red de oficina)	≤ 6 s (4G)

Estos valores son un punto de partida razonable para un equipo de gama media; se fijan definitivamente al medir el conjunto sintético de 63 mil nodos en la fase F3. Para el residente en móvil conviene una vista 2D de planta y estado de su unidad en lugar del visor 3D completo.

8. Seguridad multi-tenant y RBAC

8.1 Modelo de tenancy y alcance

Decisión recomendada: el **tenant es la organización administradora** y el **conjunto es el nodo raíz (SITE)**. Así una administradora con varios conjuntos los gestiona bajo un mismo tenant, y los permisos pueden acotarse a un subárbol (una torre, una zona, una unidad). Un rol, por tanto, es *rol + alcance*: no basta con “operator”; hay que saber “operator de la torre 3”.

8.2 Matriz de permisos propuesta

Acción	owner	admin	operator	vigilancia	viewer	residente
Ver árbol y estados	●	●	●	●	●	◐
Ver telemetría	●	●	●	◐	●	◐
Reconocer y resolver alertas	●	●	●	●	○	○
Instanciar o editar activos	●	●	●	○	○	○
Editar reglas	●	●	○	○	○	○
Ingestar modelos IFC	●	●	○	○	○	○
Generar reportes	●	●	●	◐	○	○
Gestionar usuarios	●	●	○	○	○	○
Desactivar tenant o cambiar plan	●	○	○	○	○	○

● *permitido* · ◐ *solo dentro de su alcance (subárbol asignado)* · ○ *no permitido. Propuesta sujeta a validación con la administración del conjunto.*

8.3 Reglas de oro

1. **El tenant sale del token firmado** y la membresía se verifica en cada petición; nunca desde Host, slug o parámetros del cliente.
2. **RLS en PostgreSQL como segunda barrera:** rol de aplicación sin privilegios de superusuario y *SET LOCAL app.tenant_id* por transacción (compatible con pool).
3. **Una sola vía de autenticación por canal:** cookie httpOnly + CSRF para web, Bearer para móvil y API, en una única dependencia.
4. **Refresh con rotación** y revocación por *jti*; access tokens de vida corta.
5. **Auditoría** de cambios de tenant, invitaciones, cambios de rol, instanciación y exportaciones.
6. **Pruebas de fuga generadas desde OpenAPI:** un usuario del tenant B nunca ve ni modifica datos del tenant A (ver 12.7).

8.4 Política RLS (referencia, no ejecutada)

```
ALTER TABLE twin_nodes ENABLE ROW LEVEL SECURITY;
ALTER TABLE twin_nodes FORCE  ROW LEVEL SECURITY;      -- aplica incluso al dueño de la tabla

CREATE POLICY tenant_isolation ON twin_nodes
  USING      (tenant_id = nullif(current_setting('app.tenant_id', true), '')::uuid)
  WITH CHECK (tenant_id = nullif(current_setting('app.tenant_id', true), '')::uuid);

-- Por transacción, desde la dependencia de tenant de la API:
--   SELECT set_config('app.tenant_id', :tenant_id, true);   -- true = local a la transacción
```

RLS no sustituye las comprobaciones de la API: es la red de seguridad cuando una consulta olvida el filtro. Sin app.tenant_id la política devuelve cero filas (falla cerrado).

9. Telemetría, reglas y alertas a escala

El código del motor de reglas (módulo C) no estaba en el material revisado; esta sección define los requisitos de diseño que debe cumplir para el volumen del capítulo 3.

9.1 Flujo recomendado

1. **MQTT con ACL por tenant:** tópicos `tenant/{t}/node/{id}/metric/{m}`; cada tenant solo publica y suscribe en su prefijo.
2. **Servicio de telemetría** separado de la API: valida carga útil, descarta duplicados y escribe por lotes (COPY o executemany) en la hypertable.
3. **Evaluación de reglas en memoria:** estado por (nodo, métrica) con último valor, ventana, histéresis y antirrebote; las reglas se recargan al cambiar, no por mensaje.
4. **Alertas con clave de deduplicación** (nodo, regla) y ciclo open → acknowledged → resolved; escalado por tiempo sin atender.
5. **Distribución a clientes** por Redis pub/sub y salas por tenant y subárbol; el cliente suscribe solo a lo que ve.
6. **Contrapresión:** colas acotadas, descarte controlado de muestras redundantes y métricas de retraso.

9.2 Almacenamiento y retención (punto de partida)

Capa	Resolución	Retención inicial	Mecanismo
Cruda	Muestra original	30–90 días	Hypertable con compresión.
Agregado de 1 minuto	min, max, avg, n	1 año	Agregado continuo.
Agregado de 1 hora	min, max, avg, n	Varios años	Agregado continuo.
Eventos (alertas)	Una fila por alerta	Indefinida	Tabla relacional con índices por nodo y fecha.

Los reportes de telemetría (H-05) deben leer de los agregados, no de la tabla cruda. Si el sostenido supera unos pocos miles de mensajes por segundo o se requiere alta disponibilidad del broker, conviene evaluar un broker MQTT en clúster; con el piloto, Mosquitto es suficiente.

10. Operación y producción

10.1 Objetivos de servicio propuestos

Indicador	Objetivo propuesto	Cómo se mide
Disponibilidad de la API	99,5 % mensual	Sonda externa a /readyz.
Latencia de lecturas (p95)	≤ 300 ms	Histograma por plantilla de ruta.
Latencia MQTT → pantalla (p95)	≤ 3 s	Marca de tiempo en el mensaje y en el cliente.
Pérdida de telemetría	≤ 0,1 %	Mensajes recibidos frente a filas insertadas.
Ingesta de una torre	Medir y fijar tras F2	Duración de ingest_job.
RPO / RTO	≤ 15 min / ≤ 2 h	Ensayo de restauración trimestral.

10.2 Copias de seguridad (3-2-1)

Qué	Cómo	Frecuencia	Retención
Base de datos	pgBackRest (o pg_dump -Fc) a almacenamiento externo	Diaria	30 días + mensual 12 meses
WAL	Archivado continuo para recuperación a un punto en el tiempo	Continua	7–14 días
Objetos (IFC, tiles, reportes)	Versionado del bucket + replicación a otro destino	Continua	Según política
Configuración y secretos	Gestor de secretos o repositorio cifrado	Por cambio	Histórico
Restauración	Ensayo en entorno aislado con verificación de conteos	Trimestral	Informe del ensayo

10.3 Despliegue y CI/CD

- **Proxy inverso con TLS** (Caddy, Traefik o nginx) como único puerto expuesto; la API, la base de datos y MinIO quedan en red interna.
- **Imágenes con etiqueta por SHA**, escaneo de vulnerabilidades y SBOM en CI; nunca *latest* en producción.
- **Healthchecks** y *depends_on* con condición; límites de CPU y memoria por servicio.
- **Migraciones** con patrón expandir/contraer y un job que las aplica sobre PostgreSQL real y ejecuta *alembic check*.
- **E2E de Playwright contra el stack de compose** (backend real, base real, datos semilla), no contra servidores simulados.
- **Secretos fuera del repositorio**; rotación de JWT_SECRET con ventana de convivencia de claves.
- **Protección de datos personales:** el módulo de residentes trata datos personales; validar con asesoría legal los requisitos de la normativa aplicable (consentimiento, finalidad, retención, derechos del titular).

11. Auditoría del código propuesto (módulos A–I)

Revisión por lectura de los fragmentos de código de los documentos de trabajo. Total: **52** hallazgos. Gravedad: **Crítica** = rompe el módulo o abre una fuga entre tenants; **Alta** = fallo funcional o de seguridad probable; **Media** = no escala o es frágil; **Baja** = limpieza y consistencia.

Módulo	Crítica	Alta	Media	Baja	Total
A · Ingesta IFC	0	3	7	2	12
B · Visor 3D	0	2	5	1	8
E · Catálogo de activos	0	1	2	1	4
G · Auth multi-tenant y RBAC	2	2	5	2	11
H · Reportes	0	2	5	1	8
I · Observabilidad y hardening	0	2	6	1	9
Total	**2**	**12**	**30**	**8**	**52**

Celdas coloreadas = hay al menos un hallazgo de esa gravedad en el módulo.

A · Ingesta IFC

ID	Gravedad	Hallazgo	Corrección recomendada
A-01	**Alta**	Celery serializa en JSON por defecto y la tarea recibe `content: bytes` (hasta 500 MB) a través del broker Redis: falla o satura memoria.	Subir el IFC a S3 con URL prefirmada y pasar solo `s3_key`; el worker lo descarga en streaming a disco temporal.
A-02	**Alta**	`create_shape(settings, express_id)` recibe un entero, pero la API espera una instancia de entidad. Cada fallo cae en `except` y devuelve None sin dejar rastro: la geometría quedaría vacía sin error visible.	Usar `ifc.by_id(eid)` o, mejor, `ifcopenshell.geom.iterator` multihilo; registrar cada fallo en `ingest_job_items`.
A-03	**Alta**	Los elementos contenidos (bombas, válvulas, sensores) nunca generan nodo: `_walk` solo emite referencia para tipos de `SPATIAL_TYPES` y `express_to_uuid.get(eid)` devuelve None. No hay activos ni geometría de activos.	Lista blanca de clases IFC operables → ASSET/SENSOR, hijos del contenedor espacial.
A-04	**Media**	`USE_WORLD_COORDS=True` hornea la posición global en los vértices y el visor aplica además `transform_json`: riesgo de doble transformación. IFC es Z-up y three.js es Y-up; no hay conversión.	Convención única (transform local al padre, Y-up, metros) verificada con un IFC de referencia.
A-05	**Media**	Búsqueda lineal `next(n for n in created ...)` dentro de bucles: O(n²). Con 60 mil nodos es inviable.	Diccionario uuid → nodo e inserción por lotes (COPY o `bulk_insert_mappings`).
A-06	**Media**	La geometría se exporta elemento por elemento y además se duplica: un GLB por piso y otro por activo. Doble almacenamiento y doble render.	Una sola representación: mallas fusionadas por piso y material, instancias para familias repetidas, manifiesto GlobalId → rango/instancia.
A-07	**Media**	No existe `/ingest/jobs/{id}` (el procedimiento de prueba lo usa), ni progreso, ni errores por elemento.	Tablas `ingest_jobs` e `ingest_job_items`; estados queued/running/failed/completed.
A-08	**Media**	Reingestar el mismo IFC duplica nodos: no hay clave natural. `code=GlobalId` no es único por tenant y `ifc_express_id` no es estable entre exportaciones.	Unicidad `(tenant_id, model_source_id, global_id)` y upsert; los ausentes pasan a `retired`.
A-09	**Media**	`upload_bytes` devuelve `endpoint/bucket/key` (inaccesible en un bucket privado) mientras el catálogo espera una clave: contrato inconsistente.	Guardar solo claves y firmar al servir; endpoint público distinto del interno.

ID	Gravedad	Hallazgo	Corrección recomendada
A-10	**Media**	`asyncio.run()` por tarea con un engine async con pool: errores de “loop distinto” y conexiones colgadas. boto3 síncrono bloquea el event loop en handlers async.	`NullPool` (o driver síncrono) en workers; `run_in_threadpool` para S3 en la API.
A-11	**Baja**	`IfcProject` e `IfcSite` se mapean ambos a “SITE”: nivel duplicado; un conjunto necesita tipos distintos (conjunto, zona, torre).	Taxonomía propia (sección 4.3) con mapeo explícito IFC → nodo.
A-12	**Baja**	`requirements.txt` incompleto: faltan pydantic-settings, passlib, email-validator, numpy (explícito), prometheus-client, sentry-sdk y croniter.	Lockfile (pip-tools o uv) y prueba de instalación limpia en CI.

B · Visor 3D

ID	Gravedad	Hallazgo	Corrección recomendada
B-01	**Alta**	`useChildren(null)` consulta la raíz (`enabled: parentId !== undefined` admite null). Cada NodeRenderer que no quiere hijos igualmente pide y renderiza los nodos raíz: tráfico y posible recursión.	Distinguir `undefined` (no cargar) de `null` (raíz). Ver sección 12.5.
B-02	**Alta**	`useTwinStore()` sin selector en cada nodo: cada mensaje de telemetría re-renderiza todo el árbol.	Selectores por nodo y actualizaciones agrupadas (4–10 Hz).
B-03	**Media**	Falta `@tanstack/react-query` en package.json y el `QueryClientProvider`. `zustand-temporal` no parece el paquete habitual (zundo): verificar. El historial undo registraría cada muestra de telemetría.	Declarar dependencias; limitar `temporal` al estado de edición (`partialize`).
B-04	**Media**	`CameraRig` hace lerp en cada frame y reescribe la cámara: pelea con OrbitControls (tras un foco el usuario no puede orbitar).	Transición puntual (CameraControls) que se cancela al interactuar.
B-05	**Media**	Un draw call y un GLB por activo; `scene.clone` y material clonado por resaltado sin `dispose`. No escala a decenas de miles de objetos.	Fusión por piso, `InstancedMesh`, resaltado por atributo y picking por ID.
B-06	**Media**	Eventos en un `<mesh>` que envuelve a un `<primitive>` u otras mallas: hit-testing inconsistente.	Eventos en el objeto raíz (`<group>`); raycast con BVH.
B-07	**Baja**	`TwinCanvas` recibe `siteId` y se le pasa `onNodePlaced` sin declararlo; `DropHandler` nunca se monta; `intersectPlane` devuelve null y `if (!point)` nunca falla.	Montar DropHandler dentro de `<Canvas>` y usar el valor de retorno.
B-08	**Media**	Telemetría con `socket.io-client`: FastAPI no habla Socket.IO de forma nativa.	Confirmar python-socketio o usar WebSocket nativo; salas por tenant y subárbol.

E · Catálogo de activos

ID	Gravedad	Hallazgo	Corrección recomendada
E-01	**Alta**	`instantiate` busca la plantilla solo por id (sin filtrar por tenant) y no verifica que `parent_node_id` pertenezca al tenant: referencia directa insegura entre tenants (IDOR).	Filtrar plantilla por tenant o global (NULL), validar el padre y responder 404 uniforme.
E-02	**Media**	`GET /assets` y `POST /assets/instantiate` sin `require_role`: un viewer puede crear nodos.	Matriz RBAC (sección 8); instanciar exige operator o superior.
E-03	**Media**	La posición se calcula en coordenadas de mundo y se guarda como transform del nodo; con jerarquía de transformaciones debe convertirse al espacio local del padre.	Invertir la matriz mundo del padre en cliente o convertir en servidor.
E-04	**Baja**	Subida sin validar tipo ni tamaño (cabecera `glTF`, límite) y lectura completa en memoria.	Validar cabecera y límite; URL prefirmada o streaming.

G · Auth multi-tenant y RBAC

ID	Gravedad	Hallazgo	Corrección recomendada
G-01	Crítica	`creds = await request.headers.get(...)`: `headers.get` no es awaitable → TypeError dentro de `get_current_tenant`, que usan casi todas las rutas.	Quitar el `await`; mejor reutilizar el payload ya decodificado (sección 12.1).
G-02	Crítica	Si el token no trae tenant (login sin slug, refresh), el fallback devuelve el primer segmento de `Host` sin verificar membresía. En rutas sin `require_role` (catálogo, instanciar, /auth/me) eso permite apuntar a otro tenant con un Host controlado por el cliente.	Resolver el tenant solo desde el token firmado y verificar membresía en cada petición; RLS como segunda barrera.
G-03	Alta	`get_current_user` solo lee `Authorization: Bearer`, pero `login` entrega cookies httpOnly y el frontend usa `credentials: include` sin cabecera: 401 en todo. Las pruebas con `curl -b cookies.txt` fallarían igual.	Una sola dependencia que acepte cookie (web) o Bearer (API/móvil).
G-04	Alta	`invite_user` valida el rol contra el tenant del token, no el de la ruta; un admin puede invitar como `owner`; el `try/except` no captura la violación de unicidad porque ocurre en `commit`.	Rol validado sobre el tenant de la ruta, prohibir asignar un rol mayor al propio, capturar IntegrityError.
G-05	Media	`RequireRole` usa `user.role`, pero `/auth/me` no devuelve el rol: el guard siempre redirige.	Incluir `role` en /auth/me; el guard cliente es solo UX, no seguridad.
G-06	Media	Refresh sin rotación ni revocación y sin conservar tenant ni rol; logout solo borra cookies.	`jti` en Redis/BD, rotación en cada uso y lista de revocación.
G-07	Media	Sin límite de intentos de login ni política de contraseñas; invitados con contraseña aleatoria y sin flujo de restablecimiento (queda como TODO).	Límite por IP+email, bloqueo progresivo, token de invitación con expiración.
G-08	Media	`block_inactive_tenants` lee `request.state.tenant_id`, que nunca se asigna: middleware inerte (y consultaría la BD en cada petición).	Validar `active` dentro de la dependencia de tenant con caché corta en Redis.
G-09	Media	Cookies `secure=True` impiden probar con curl por http; no hay CSRF documentado más allá de SameSite=Lax.	Perfiles dev/prod; token CSRF (doble envío) si hay subdominios o formularios cross-site.
G-10	Baja	`JWT_SECRET`, `SENTRY_DSN`, `ENV`, `CORS_ORIGINS` y `ALLOWED_HOSTS` se usan pero no están en `Settings`.	Completar Settings y validar al arrancar (fallar rápido).
G-11	Baja	`X-Tenant-Id` permite al superadmin cambiar de tenant sin dejar rastro.	Registrar cada cambio en `audit_log`.

H · Reportes

ID	Gravedad	Hallazgo	Corrección recomendada
H-01	Alta	`to_csv` escribe `str` en `BytesIO` con `csv.DictWriter`: TypeError (reproducido en el entorno de trabajo).	`StringIO` y `encode("utf-8-sig")` (sección 12.2, probado).
H-02	Alta	El scheduler genera todos los reportes habilitados cada hora e ignora `schedule_cron`.	Evaluar vencimiento con croniter contra `last_run_at`; idempotencia por (reporte, ventana).
H-03	Media	`assets_status` con `node_id` filtra solo hijos directos; en un conjunto necesita descendientes.	Reutilizar la CTE recursiva de `hierarchy_queries` o path con ltree.
H-04	Media	Inyección de fórmulas en CSV/XLSX: celdas de texto que empiezan por `= + - @` se ejecutan al abrirse en Excel.	Neutralizar con prefijo en cadenas (incluido en 12.2).
H-05	Media	`telemetry_summary` exige `date_from`/`date_to` (KeyError → 500) y agrega datos crudos.	Validar con Pydantic y usar agregados continuos de TimescaleDB.
H-06	Media	Generación síncrona dentro de la petición; el PDF se trunca a 500 filas; cada descarga guarda una copia en MinIO sin retención.	Tarea Celery para reportes grandes con estado y descarga prefirmada; ciclo de vida del bucket.

ID	Gravedad	Hallazgo	Corrección recomendada
H-07	**Media**	Los enlaces prefirmados se firman con el endpoint interno (`http://minio:9000`): inaccesibles desde navegador o correo.	Endpoint público separado (`S3_PUBLIC_ENDPOINT`) para firmar.
H-08	**Baja**	weasyprint se declara pero no se usa; `scalar_one()` sin manejo de tenant inexistente.	Quitar la dependencia pesada y manejar el caso.

I · Observabilidad y hardening

ID	Gravedad	Hallazgo	Corrección recomendada
I-01	**Alta**	`app.add_route("/metrics", metrics_endpoint)` con una función sin parámetro `request`: Starlette la invoca con la petición → TypeError.	Firma `async def metrics_endpoint(request)` (sección 12.6).
I-02	**Alta**	Servicio de backup: en Alpine el paquete `mc` es Midnight Commander (el cliente de MinIO se instala de otra forma; verificar); cron no hereda variables de entorno; `/backups` no es un volumen (copias perdidas al recrear); el `pg_restore` apunta a una ruta inexistente en el contenedor postgres.	Imagen dedicada, destino remoto o volumen, script con entorno explícito y restauración probada automáticamente.
I-03	**Media**	Backups solo de BD: sin objetos de MinIO (IFC, tiles), sin archivado de WAL (RPO de 24 h) y sin restauración ensayada.	pgBackRest o WAL-G + versionado/replicación de bucket; ensayo periódico con RTO/RPO medidos.
I-04	**Media**	Rate limit por `request.client.host`: tras un proxy todos comparten IP; ventana fija permite ráfaga doble; falta importar `JSONResponse` en ese fragmento; login sin tope estricto.	Cabeceras de proxy de confianza, límites por usuario/tenant, ventana deslizante o token bucket, tope bajo en /auth.
I-05	**Media**	Métricas etiquetadas por `request.url.path` (cardinalidad) con parche `/dynamic`; `/metrics` público; varios workers sin modo multiproceso.	Etiquetar por plantilla de ruta; proteger /metrics; `PROMETHEUS_MULTIPROC_DIR`.
I-06	**Media**	CSP con `script-src 'unsafe-inline'` y `connect-src https: wss:` demasiado abierta.	Nonces o hashes (el Command Center ya calcula hashes) y lista explícita de orígenes.
I-07	**Media**	Compose de producción: sin proxy TLS (expone 8000 y 3000), etiquetas `latest`, sin healthchecks ni `depends_on` con condición, sin límites de recursos, autenticación de Mosquitto no visible.	Caddy/Traefik/nginx con TLS, tags por SHA, healthchecks, límites y ACL MQTT por tenant.
I-08	**Media**	CI sin migraciones sobre Postgres, sin escaneo de vulnerabilidades ni SBOM, imagen solo `latest`, Playwright sin backend real.	Job de migración + `alembic check`, Trivy, tags por SHA, E2E contra el stack de compose.
I-09	**Baja**	`/health` mezcla liveness y readiness y usa boto3 síncrono.	`/livez` y `/readyz` con timeouts.

12. Correcciones de referencia

> **Estado de estos fragmentos**
> Solo 12.2 (CSV) se ejecutó y pasó su prueba en el entorno de trabajo. El resto es código de referencia **sin ejecutar**: debe integrarse y probarse en F0–F1. Ajustar nombres de módulos y modelos a los del repositorio.

12.1 Identidad y tenant (cierra G-01, G-02, G-03, G-08, G-11)

```
async def get_current_user(request: Request, creds=Depends(bearer), db: AsyncSession = Depends(get_db)) -> User:
    token = creds.credentials if creds else request.cookies.get("access_token")
    if not token:
        raise HTTPException(401, "Falta token")
    payload = decode_token(token)
    if payload.get("type") == "refresh":
        raise HTTPException(401, "Token inválido")
    user = (await db.execute(select(User).where(User.id == payload["sub"]))).scalar_one_or_none()
    if not user or not user.is_active:
        raise HTTPException(401, "Usuario inválido")
    request.state.token_payload = payload
    return user


async def get_tenant_ctx(request: Request, user: User = Depends(get_current_user),
                         db: AsyncSession = Depends(get_db)) -> TenantCtx:
    tenant_id = request.state.token_payload.get("tenant_id")
    if user.is_superadmin and (hdr := request.headers.get("X-Tenant-Id")):
        await audit(db, user.id, "superadmin_tenant_switch", hdr)
        tenant_id = hdr
    if not tenant_id:
        raise HTTPException(400, "Token sin tenant: seleccione uno")
    role = await membership_role(db, user.id, tenant_id)        # None si no es miembro
    if role is None and not user.is_superadmin:
        raise HTTPException(403, "Sin acceso al tenant")
    if not await tenant_is_active(tenant_id):                    # caché corta en Redis
        raise HTTPException(403, "Tenant inactivo")
    await db.execute(text("SELECT set_config('app.tenant_id', :t, true)"), {"t": str(tenant_id)})
    return TenantCtx(id=str(tenant_id), role=role or "superadmin")
```

12.2 CSV seguro (cierra H-01, H-04) · probado

```
import csv, io
_FORMULA = ("=", "+", "-", "@", "\t", "\r")

def _safe(v):
    if isinstance(v, str) and v.startswith(_FORMULA):
        return "'" + v                    # neutraliza fórmulas; los números no se tocan
    return "" if v is None else v

def to_csv(rows):
    if not rows:
        return b""
    buf = io.StringIO(newline="")
    w = csv.DictWriter(buf, fieldnames=list(rows[0].keys()))
    w.writeheader()
    w.writerows({k: _safe(v) for k, v in r.items()} for r in rows)
    return buf.getvalue().encode("utf-8-sig")   # BOM: Excel respeta tildes y eñes
```

12.3 Reportes programados que respetan el cron (cierra H-02)

```
from croniter import croniter

def is_due(cron: str, last_run: datetime | None, created_at: datetime, now: datetime) -> bool:
    base = last_run or created_at
    return croniter(cron, base).get_next(datetime) <= now

@celery_app.task
def dispatch_scheduled_reports():
    async def _run():
        async with async_session_factory() as db:
            now = datetime.now(timezone.utc)
```

```
            for rep in (await db.execute(select(ScheduledReport).where(ScheduledReport.enabled))).scalars():
                if is_due(rep.schedule_cron, rep.last_run_at, rep.created_at, now):
                    run_report.delay(str(rep.id))          # una tarea por reporte; idempotente por ventana
                    rep.last_run_at = now
            await db.commit()
    asyncio.run(_run())
```

12.4 Ingesta por clave de S3 y engine apto para workers (cierra A-01, A-10)

```
# db_worker.py
engine = create_async_engine(settings.DATABASE_URL, poolclass=NullPool)   # sin pool entre event loops
worker_session = async_sessionmaker(engine, expire_on_commit=False)

# api.py : la API entrega URL prefirmada y registra el job
@router.post("/ifc/upload-url")
async def upload_url(ctx: TenantCtx = Depends(require_role("owner", "admin"))):
    key = f"tenant/{ctx.id}/ifc/incoming/{uuid4()}.ifc"
    url = await run_in_threadpool(s3.generate_presigned_url, "put_object",
                                  Params={"Bucket": settings.S3_BUCKET, "Key": key}, ExpiresIn=900)
    return {"key": key, "url": url}

# tasks.py : solo viaja la clave
@celery_app.task(bind=True, max_retries=3, acks_late=True)
def ingest_ifc_task(self, s3_key: str, tenant_id: str, job_id: str):
    ...   # descarga a disco temporal, procesa por etapas y actualiza ingest_jobs
```

12.5 Hook de hijos y selectores por nodo (cierra B-01, B-02)

```
// api.ts : undefined = no cargar, null = raíz
export function useChildren(parentId: string | null | undefined) {
  return useQuery({
    queryKey: ["twin", "children", parentId ?? "root"],
    queryFn: () => fetchChildren(parentId ?? null),
    enabled: parentId !== undefined,
    staleTime: 60_000,
  });
}

// NodeRenderer.tsx
const { data: children } = useChildren(shouldLoadChildren ? node.id : undefined);
const isSelected = useTwinStore((s) => s.selectedNodeId === node.id);
const isHovered  = useTwinStore((s) => s.hoveredNodeId === node.id);
const alertLevel = useTwinStore((s) => s.alerts[node.id]);
```

12.6 Métricas (cierra I-01, I-05)

```
from starlette.requests import Request

async def metrics_endpoint(request: Request) -> Response:          # la firma recibe la petición
    return Response(generate_latest(), media_type=CONTENT_TYPE_LATEST)

async def metrics_middleware(request: Request, call_next):
    start = time.perf_counter()
    response = await call_next(request)
    route = request.scope.get("route")                              # plantilla: /twin/nodes/{id}
    path = route.path if route else "unmatched"                     # verificar con la versión instalada
    http_requests_total.labels(request.method, path, response.status_code).inc()
    http_request_duration.labels(request.method, path).observe(time.perf_counter() - start)
    return response
```

12.7 Prueba automática de fuga entre tenants

```
ROUTES = [("GET", "/assets"), ("GET", "/twin/nodes?root=true"), ("GET", "/reports/scheduled"),
          ("GET", "/auth/me")]

@pytest.mark.parametrize("method,path", ROUTES)
async def test_no_cross_tenant_leak(client, token_b, marker_a, method, path):
    # marker_a: texto único sembrado solo en datos del tenant A
    for extra in ({}, {"Host": "tenant-a.example.com"}, {"X-Tenant-Id": TENANT_A_ID}):
        r = await client.request(method, path, headers={"Authorization": f"Bearer {token_b}", **extra})
        assert marker_a not in r.text, (method, path, extra)
```

13. Hoja de ruta por fases

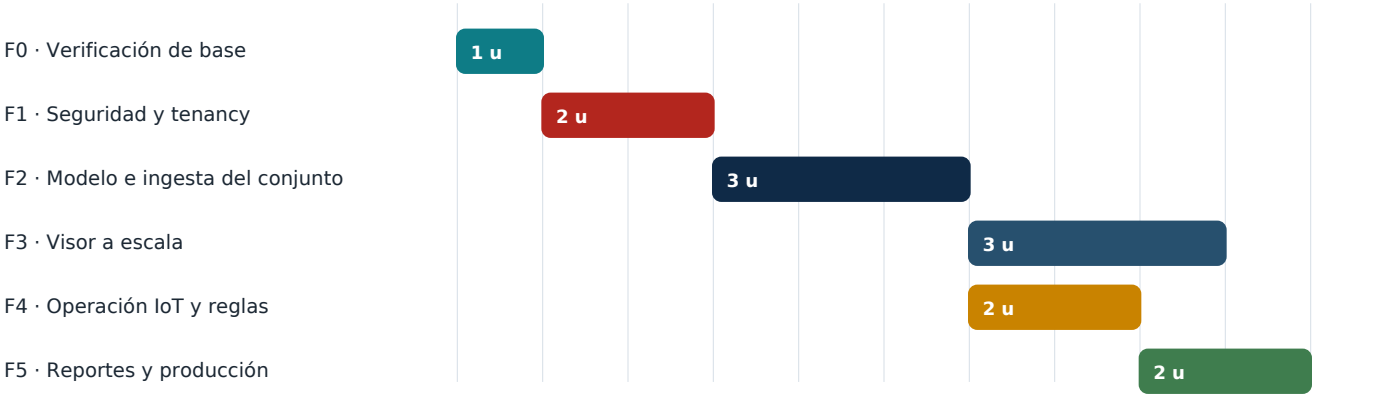


Figura 4. Esfuerzo relativo y orden de las fases. No es un calendario: la duración depende del equipo y de la disponibilidad de IFC reales.

Fase	Talla	Entregables	Criterio de salida medible
F0 · Verificación de base	S	Levantar PostgreSQL/TimescaleDB, Redis, MinIO y Mosquitto con compose; ejecutar migraciones y las 200 pruebas contra la base real; correr las 7 pruebas SQL de jerarquía en PostgreSQL; instalar y compilar los frontends React; probar el Command Center contra el backend real.	CI en verde con PostgreSQL real; E2E contra backend levantado; plan de ejecución (EXPLAIN) de las consultas de jerarquía.
F1 · Seguridad y tenancy	M	Cerrar G-01 a G-11 y E-01, E-02; una dependencia de identidad (cookie o Bearer); RLS; matriz RBAC con alcance por subárbol; auditoría; límites de login.	Suite de fuga entre tenants al 100 %; cero hallazgos críticos o altos abiertos en auth y catálogo.
F2 · Modelo e ingesta del conjunto	L	model_sources, path/ltree, rollups, ingest_jobs; upsert idempotente; lista blanca de activos; geometría con iterador, fusión y manifiesto; ingesta de tres torres reales.	Dos ingestas consecutivas sin duplicados; árbol del conjunto navegable por API; tiempo de ingesta por torre medido.
F3 · Visor a escala	L	Tiles por torre y piso, LOD, instancing, picking por ID, selectores, transiciones de cámara; generador de conjunto sintético de 63 mil nodos.	Presupuesto del cap. 7 cumplido en el equipo de referencia; sin fugas de memoria al navegar 30 minutos.
F4 · Operación IoT y reglas	M	Servicio de telemetría por lotes, reglas con estado en memoria, alertas deduplicadas, salas WebSocket por subárbol, ACL MQTT por tenant, prueba de carga.	Carga sostenida de 500 a 3.000 mensajes/s sin pérdida; p95 MQTT → pantalla ≤ 3 s.
F5 · Reportes y producción	M	Reportes asíncronos con agregados, CSV seguro, scheduler con cron; proxy TLS; backups con WAL y bucket replicado; observabilidad; runbooks.	Ensayo de restauración con RPO/RTO medidos; lista de salida a producción (anexo A) al 100 %.

Orden recomendado
F0 → F1 → F2 son secuenciales: no tiene sentido medir el visor (F3) sin una ingesta confiable, ni abrir el sistema a más usuarios sin cerrar la seguridad. F3 y F4 pueden avanzar en paralelo una vez termina F2.

14. Riesgos y decisiones abiertas

14.1 Registro de riesgos

Riesgo	Prob.	Impacto	Mitigación
Los IFC reales son pesados o de baja calidad (sin pisos, unidades erróneas, GlobalId repetidos).	Alta	Alto	Validación al subir, informe de calidad por archivo y casos límite del cap. 6 cubiertos por pruebas.
El visor no alcanza el presupuesto en equipos modestos.	Media	Alto	LOD agresivo, vista 2D para móvil, medición temprana en F3.
Deriva entre resúmenes (✓) y estado real del código.	Alta	Medio	Escala de madurez L0–L4 y criterio de salida por fase.
Fuga de datos entre tenants.	Media	Muy alto	Token + membresía + RLS + pruebas automáticas (F1).
Telemetría real distinta de la supuesta (más sensores o frecuencia).	Media	Medio	Prueba de carga con parámetros configurables y agregados desde el inicio.
Costo de almacenamiento de geometría y telemetría.	Media	Medio	Compresión, retención por capas y ciclo de vida de buckets.
Dependencia de una sola persona para el conocimiento del sistema.	Media	Alto	Runbooks, ADR cortos y documentación mantenida en el repositorio.

14.2 Decisiones por tomar

Decisión	Recomendación	Alternativa y cuándo considerarla
Monolito modular o microservicios	Monolito modular + workers.	Separar servicios solo si un módulo exige escalado o ciclo de despliegue propio.
Modelo de jerarquía	parent_id + ltree + rollups.	Tabla de cierre si las lecturas de subárbol dominan y hay poca mutación.
Formato de geometría	glTF por tiles con manifiesto propio.	Formato de fragmentos o 3D Tiles si los modelos superan lo manejable.
Autenticación	Cookie httpOnly + CSRF (web) y Bearer (móvil/API) en una dependencia.	Solo Bearer si no hay frontend web propio.
Definición de tenant	Organización administradora; el conjunto es nodo raíz.	Un tenant por conjunto si cada copropiedad contrata por separado.
Broker MQTT	Mosquitto para el piloto.	Broker en clúster con sostenido de miles de mensajes/s o requisito de alta disponibilidad.
Serie temporal	TimescaleDB (hypertables y agregados continuos).	Particionado nativo de PostgreSQL si no se puede operar la extensión.
Reportes	Sincrónicos hasta un umbral de filas; asíncronos por encima.	Siempre asíncronos si el volumen es alto desde el inicio.

Anexo A. Lista de salida a producción

- [ ] Migraciones aplicadas y verificadas sobre PostgreSQL real; *alembic check* sin diferencias.
- [ ] RLS activo y probado; rol de aplicación sin privilegios de superusuario.
- [ ] Suite de fuga entre tenants al 100 % en CI.
- [ ] Sin hallazgos críticos ni altos abiertos (cap. 11).
- [ ] Un solo mecanismo de autenticación por canal; refresh con rotación; límites de intentos de login.
- [ ] TLS en el proxy; puertos internos no expuestos; cabeceras de seguridad sin *unsafe-inline* en scripts.
- [ ] Imágenes por SHA, escaneadas, con SBOM; secretos fuera del repositorio.
- [ ] Backups de base de datos, WAL y objetos; ensayo de restauración con RPO y RTO medidos y documentados.
- [ ] Métricas, trazas y alertas operativas; paneles por módulo; sondas /livez y /readyz.
- [ ] Prueba de carga de telemetría y prueba de rendimiento del visor con el conjunto sintético de 63 mil nodos.
- [ ] Tres torres reales ingestadas dos veces sin duplicados; informe de calidad por archivo.
- [ ] Runbooks: restauración, rotación de secretos, reingesta, reversión de despliegue.
- [ ] Revisión legal del tratamiento de datos personales de residentes.

Anexo B. Glosario

Término	Definición
BIM	Modelado de información de construcción: modelo digital con geometría y datos de los elementos de un edificio.
IFC	Formato abierto de intercambio BIM. Cada elemento tiene un GlobalId estable entre exportaciones.
GlobalId / express id	El primero identifica un elemento de forma estable; el segundo es un número interno del archivo que cambia al reexportar.
Modelo federado	Conjunto de modelos de distintas disciplinas (arquitectura, estructura, MEP) que se combinan sin fusionarse en un solo archivo.
glTF / GLB	Formato de geometría 3D para la web; GLB es su versión en un solo archivo binario.
Tile	Fragmento de geometría que se carga bajo demanda según el nivel de detalle.
LOD	Nivel de detalle: versión más simple o más rica de la misma geometría según la distancia o el contexto.
Instancing	Dibujar muchas copias de la misma malla con una sola llamada al GPU.
ltree	Extensión de PostgreSQL para rutas jerárquicas con índices de subárbol.
RLS	Seguridad a nivel de fila: la base de datos filtra filas según una política (por ejemplo, el tenant).
Hypertable	Tabla de TimescaleDB particionada por tiempo, pensada para series temporales.
Agregado continuo	Resumen de una serie temporal que la base de datos mantiene actualizado de forma incremental.
MQTT	Protocolo de mensajería ligero, habitual en IoT, basado en tópicos y suscripciones.
IDOR	Referencia directa insegura a objetos: acceder a un recurso ajeno cambiando su identificador.
RPO / RTO	Pérdida máxima de datos tolerable / tiempo máximo para restablecer el servicio.
SLO	Objetivo de nivel de servicio medible, por ejemplo latencia p95 o disponibilidad.